

Phishing Attack Detection using Machine Learning: Reproduction and Critical Evaluation

Phishing Attack Detection using Machine Learning: Reproduction and Critical Evaluation

Final Project — Data Science in Cybersecurity

Selected Source:

RimTouny / Phishing-Attack-Detection-using-Machine-Learning

<https://github.com/RimTouny/Phishing-Attack-Detection-using-Machine-Learning>

Dataset Used:

dataset_phishing.csv

Student Name:

Arkan Issa - 212932206

Table of Contents:

1. Executive Summary
2. Summary of the Source
3. Critical Evaluation
4. Feature Engineering Analysis
5. Reproducibility Analysis
6. Experimental Results
7. Evaluation and Error Analysis
8. Conclusions
9. Summing It Up

Executive Summary

This notebook presents an adapted reproduction and critical evaluation of the GitHub project “Phishing Attack Detection using Machine Learning”. The selected project addresses a cybersecurity binary classification problem: detecting whether a URL is phishing or benign. This problem is important because phishing attacks are commonly used to steal passwords, personal information, and financial credentials from users.

The original project proposes a machine learning-based approach using URL-based features. The author describes extracting features such as URL length, domain length, number of digits, number of special characters, Boolean indicators such as IP address usage or URL shortening, and calculation-based features such as entropy and malicious probability. The original project also describes the use of multiple datasets, exploratory data analysis, feature engineering, SMOTE, several classification models, and ensemble methods such as stacking and soft voting.

A major issue identified during the critical evaluation is reproducibility. The original notebook loads the processed dataset from a local Windows path, and the processed CSV file was not directly available in the repository. Therefore, the original experiment could not be reproduced exactly from a clean clone of the repository.

Because of this limitation, this notebook performs an adapted reproduction using the approved `dataset_phishing.csv` dataset. The adapted dataset contains 11,430 URL samples and 89 original columns. The target column is `status`, with two classes: legitimate and phishing. The target was encoded into numerical labels, where legitimate was mapped to 0 and phishing was mapped to 1.

The notebook includes data loading, data inspection, exploratory data analysis, feature engineering, preprocessing, model training, evaluation, threshold analysis, feature importance, cross-validation, and error analysis. Several machine learning models were trained and compared, including Logistic Regression, Decision Tree, Random Forest, Extra Trees, Gradient Boosting, and K-Nearest Neighbors.

The best-performing model was Random Forest. It achieved accuracy of 0.9624, phishing recall of 0.9676, phishing F1-score of 0.9626, and ROC-AUC of 0.9934. The confusion matrix showed 1,094 correctly classified legitimate URLs, 1,106 correctly classified phishing URLs, 49 false positives, and 37 false negatives.

The results support the general idea that machine learning can be useful for phishing URL detection when meaningful URL and webpage-based features are available. However, the results should be interpreted carefully because the original work had reproducibility limitations, the adapted dataset is balanced, and temporal generalization could not be evaluated due to the absence of timestamp features.

1. Summary of the Source

The selected source is the GitHub project “**Phishing Attack Detection using Machine Learning**” by RimTouny. The project addresses a binary classification problem in cybersecurity: determining whether a given URL is **phishing** or **benign**.

The problem being addressed is important because phishing attacks are a common cyber threat. Attackers use fake or malicious URLs to deceive users into visiting fraudulent websites and revealing sensitive information such as passwords, personal details, or financial credentials. Therefore, an automated phishing detection system can help reduce user exposure to malicious websites and support cybersecurity defense.

The proposed solution is based on machine learning. Instead of manually checking each URL, the author extracts URL-based features and trains classification models to distinguish between phishing and benign URLs. The independent variables are divided into several categories. First, there are length and count features, such as URL length, domain length, number of digits, number of letters, path components, and special symbols. Second, there are Boolean features, such as whether the URL contains an IP address, uses redirection, is shortened, is encoded, or has suspicious characteristics. Third, there are calculation-based features, such as entropy, ratios of special characters and digits, and malicious probability.

According to the project README, the original dataset was created by combining several URL datasets, including the PhishStorm URL dataset, the ISCX-URL2016 dataset, and a Malicious URL dataset. The project description states that the final dataset contains approximately 737,000 URLs. The target variable is the URL label, where 1 represents phishing and 0 represents benign.

The methodology of the original project includes several stages. The author first concatenated multiple datasets and extracted URL-based features. Then, exploratory data analysis was performed using descriptive statistics, distribution plots, boxplots, violin plots, histograms, and correlation matrices. After that, the project applied data cleaning and feature engineering, including handling null values, duplicate rows, repeated maximum values, and feature selection. The data was split into training and testing sets using an 80/20 split. The author also used SMOTE to address class imbalance in the training data.

For modeling, the original project trained and compared several classification models, including Logistic Regression, Support Vector Machine, Decision Tree, Random Forest, XGBoost, and other models using LazyClassifier. The author also explored ensemble methods, including stacking and soft voting classifiers, in order to combine strong models and improve final performance. Model performance was evaluated using confusion matrices, F1 scores, classification reports, and other classification metrics.

In this final project, the original source is used as the methodological basis for an adapted reproduction and critical evaluation. Because the original processed CSV was not directly available, the experimental part of this work uses an approved phishing dataset.

2. Critical Evaluation

The main claim of the original project is that **machine learning can be used effectively to detect phishing URLs and improve cybersecurity defense**. The author presents the project as an AI-based phishing detection system trained on a large and diverse dataset of URLs. The project claims that URL-based features, combined with machine learning classifiers and ensemble methods, can distinguish between phishing and benign URLs.

This claim is reasonable because phishing URLs often contain suspicious patterns that can be captured using engineered features. For example, phishing URLs may be unusually long, contain many digits or special characters, use redirection, include suspicious domains, or contain encoded or shortened links. Therefore, using URL-based features for phishing detection is an appropriate and relevant approach.

For example, a normal banking URL may look simple and trustworthy, such as <https://www.bank.com/login>. In contrast, a suspicious phishing URL may contain additional security-related words, numbers, unusual structure, or an uncommon domain, such as <http://bank-login-secure-update-123.xyz>. This illustrates why URL-based features such as URL length, number of digits, number of special characters, domain structure, and suspicious keywords can be useful for machine learning-based phishing detection.

The evidence presented by the author partially supports the claim. The README describes a full machine learning pipeline, including dataset concatenation, feature extraction, exploratory data analysis, data cleaning, feature selection, SMOTE for class imbalance, model training, and performance evaluation. The author also reports the use of several classification models and ensemble methods such as stacking and soft voting. This shows that the original project attempts to compare multiple models rather than relying on a single classifier.

However, there are also important limitations. First, the reproducibility of the original project is limited. Although the README states that the notebook was executed successfully in Google Colab, the processed CSV file used by the original notebook was not directly available in the repository during this reproduction attempt. This means that the original results cannot be fully reproduced exactly without access to the same processed dataset and preprocessing steps.

Second, some parts of the feature engineering process may need further explanation. The README mentions features such as `malicious_probability`, entropy, ratios, and suspicious URL indicators. Some of these features are useful, but others may introduce a risk of data leakage if they are computed using information that is too close to the target label. For example, if `malicious_probability` was derived from an external detector or blacklist, it may make the model appear stronger than it would be in a real deployment scenario.

Third, the evaluation methodology is appropriate in general, but it could be stronger. The use of train/test split, confusion matrices, F1 scores, classification reports, and ensemble methods is suitable for a binary classification task. However, phishing detection is a cybersecurity problem where accuracy alone is not sufficient. False negatives are especially dangerous because they represent phishing URLs that are incorrectly classified as benign. Therefore, the evaluation should emphasize recall, F1-score, F-beta score, MCC, ROC-AUC, and detailed error analysis.

Another limitation is the lack of temporal evaluation. Phishing attacks change over time, and attackers constantly adapt their URLs and techniques. If the data is split randomly into training and testing sets, the model may perform well on similar samples but may not generalize as well to future phishing campaigns. A time-based split or evaluation on a more recent external dataset would provide stronger evidence for real-world robustness.

The use of multiple datasets is a strength because it can increase diversity. However, combining datasets from different sources can also introduce problems such as duplicate URLs, inconsistent labels, sampling bias, and differences in collection methods. The original project would be stronger if it provided a more detailed discussion of how these issues were handled.

Overall, the author's conclusion that machine learning is useful for phishing URL detection is justified in a general sense. The methodology is relevant, and the use of URL-based features and multiple classifiers is appropriate. However, the strength of the original conclusions is limited by reproducibility issues, possible feature leakage, unclear preprocessing details, and the absence of temporal or external validation. Therefore, the project is useful as a methodological basis, but its claims should be treated carefully and validated through an independent adapted reproduction.

These limitations motivated the adapted reproduction performed in this project. Since the original processed CSV file was not directly available, the experimental part of this work uses an approved phishing dataset to test the same general idea: phishing URL detection using URL and webpage-based features with machine learning models. Therefore, this work does not treat the original claims as automatically proven, but evaluates the methodology through an independent adapted experiment.

3. Feature Engineering Analysis

Feature engineering was an important part of both the original project and the adapted reproduction performed in this work. In the original project, the author extracted URL-based features from raw URLs. These features included length and count features, Boolean features, and calculation-based features. Examples include URL length, domain length, number of digits, number of letters, path components, special characters, whether the URL contains an IP address, whether it uses redirection, whether it is shortened, entropy, and ratios of digits or special characters.

This feature engineering process is meaningful for phishing detection because phishing URLs often have structural patterns that differ from legitimate URLs. For example, phishing URLs may be longer, contain unusual characters, use IP addresses instead of domain names, include suspicious keywords, use shortened links, or contain many digits and symbols. From a cybersecurity perspective, these features represent signals that attackers may leave when trying to imitate legitimate websites or hide malicious destinations. From a mathematical perspective, the purpose of feature engineering is to convert raw URL text into numerical variables that machine learning models can use for classification.

In the adapted experiment, the dataset already contained engineered URL and webpage-based features. Therefore, the main feature engineering work was not raw URL parsing, but preprocessing, encoding, feature selection, and redundancy reduction. The target column status was encoded into a numeric label, where legitimate was mapped to 0 and phishing was mapped to 1. This encoding was necessary because machine learning classifiers require numerical target labels for supervised learning.

The original string column url was removed from the model input because it is raw text and was not directly suitable for the numerical models used in this experiment. The original target column status and the encoded target column label were also removed from the feature matrix to avoid target leakage. Only numeric features were kept for modeling. This ensured that the models were trained on measurable URL and webpage properties rather than on the answer itself.

Feature scaling was applied only to models that require it. In this project, StandardScaler was used inside pipelines for Logistic Regression and K-Nearest Neighbors. Logistic Regression is affected by the scale of features because it learns coefficients over numerical inputs, and KNN is affected by feature scale because it relies on distance calculations. Without scaling, features with large numeric ranges could dominate the model. In contrast, tree-based models such as Decision Tree, Random Forest, Extra Trees, and Gradient Boosting do not require scaling because they split the data using feature thresholds.

Feature selection was also applied. Constant columns and duplicated features were checked and removed when necessary. This is important because features that contain only one value cannot help the model distinguish between phishing and legitimate URLs. Duplicated features add redundancy and may make the model less interpretable without adding new information. Near-constant or highly repetitive features are also problematic because they carry little discriminative value.

The original project used a similar idea by removing features where the most frequent value appeared in more than 90% of the rows. This approach is reasonable because features with overwhelmingly repeated values usually provide little useful information. However, this type of feature selection should be applied carefully. In cybersecurity, rare values can sometimes be important indicators of attacks. Therefore, a feature should not be removed only because it is rare; it should be evaluated in relation to the target and its cybersecurity meaning.

No logarithmic transformation, Box–Cox transformation, one-hot encoding, target encoding, or dimensionality reduction was applied in the adapted experiment. Logarithmic and Box–Cox transformations were not used because the models included tree-based classifiers that can handle non-normal and skewed feature distributions reasonably well. One-hot encoding was not needed because the selected model inputs were numeric. Dimensionality reduction such as PCA was not applied because the number of features was manageable and interpretability was important. Keeping the original features made it easier to explain which URL and webpage characteristics were related to phishing.

Redundancy in the system can be detected in several ways. Exact duplicate columns can be found by comparing feature values directly. Near-constant features can be detected using the number of unique values or the percentage of the most common value. Highly correlated features can be detected using a correlation matrix. Another way to detect redundancy is to compare feature importance or use permutation importance to see whether some features add little predictive value. If redundancy is found, it can be handled by removing duplicated or constant features, removing one feature from highly correlated pairs, using regularization, or applying dimensionality reduction when interpretability is less important.

The feature engineering process was meaningful because it connected mathematical modeling with cybersecurity intuition. Mathematically, the process transformed raw or semi-raw URL information into structured numerical features that classifiers can learn from. In cybersecurity terms, these features represent suspicious behaviors or characteristics of malicious URLs. For example, a URL with many digits, unusual symbols, low reputation, weak web traffic, or suspicious domain patterns may be more likely to be phishing.

In the adapted experiment, important features included reputation and webpage-related indicators such as `google_index`, `page_rank`, `web_traffic`, `nb_hyperlinks`, and hyperlink ratios. These features are meaningful because legitimate websites are more likely to have stable reputation signals, indexing, traffic, and normal link structure, while phishing sites may be new, poorly indexed, low reputation, or structurally abnormal.

Additional features could potentially improve performance. These include domain registration age from WHOIS data, SSL/TLS certificate information, certificate issuer, DNS record features, domain creation date, domain expiration date, number of redirects, page content features, login form indicators, JavaScript behavior, brand impersonation keywords, blacklist or reputation features, and temporal features such as collection date. Temporal features would be especially useful because phishing tactics change over time, and a time-based evaluation could test whether the model generalizes to future phishing campaigns.

Overall, the feature engineering process in the original project was meaningful and appropriate for phishing URL detection. However, some features require careful interpretation because they may introduce data leakage if they are computed using external information that is too close to the target label. In the adapted reproduction, feature engineering focused on safe preprocessing, encoding, scaling where necessary, removal of redundant features, and preserving interpretability.

```

Preprocessing info:
{'constant_columns': ['nb_or',
  'ratio_nullHyperlinks',
  'ratio_intRedirection',
  'ratio_intErrors',
  'submit_email',
  'sfh'],
 'duplicated_columns_removed': [],
 'final_feature_count': 81}

```

Figure 1: Feature preprocessing output from the adapted notebook. The output shows that several constant or non-informative columns were removed, no duplicated columns were removed, and the final feature matrix contained 81 numerical features used for model training.

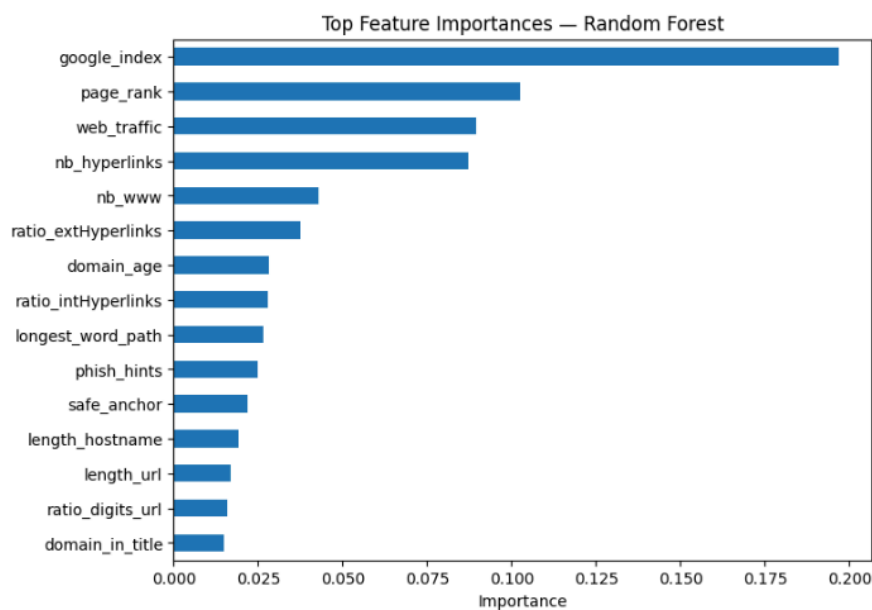


Figure 2: Top feature importances of the Random Forest model. The most important features were mainly reputation and webpage-related indicators, such as google_index, page_rank, web_traffic, and nb_hyperlinks. This supports the cybersecurity intuition that phishing websites often have weaker reputation signals, lower visibility, or abnormal webpage/link structures compared to legitimate websites.

	count	meaning
label		
0	5715	legitimate
1	5715	phishing

Figure 3: Target encoding and class distribution. The original target values were encoded into numerical labels, where 0 represents legitimate URLs and 1 represents phishing URLs. The table also shows that the dataset is balanced, with 5,715 samples in each class.

4. Reproducibility Analysis

The reproducibility of the original project is limited. The README states that the code was executed and tested successfully in a Google Colab environment and lists the required libraries, including scikit-learn, pandas, and matplotlib. However, during the reproduction attempt, the original processed CSV file used by the notebook was not directly available in the repository.

This creates an important reproducibility issue. Even if the notebook code is available, the experiment cannot be fully reproduced unless the same dataset and preprocessing steps are also available. In the original notebook, the data appears to be loaded from a local file path rather than from a public or relative path inside the repository. As a result, another user cannot simply clone the repository and run the notebook from start to finish.

Another limitation is the possibility of hidden preprocessing steps. The README explains that the author concatenated several datasets and extracted URL-based features, but the final processed dataset used for training was not directly accessible. Therefore, it is not fully clear whether all feature extraction, cleaning, filtering, duplicate removal, and label preparation steps can be reconstructed exactly. This affects the reliability of reproducing the reported results.

The required dependencies are mostly standard Python data science libraries, so the software environment itself is not the main problem. The main reproducibility weakness is the missing processed data file and the lack of a fully self-contained execution pipeline. A stronger repository would include the raw or processed dataset, a clear data preparation script, a requirements.txt file, and instructions that allow the notebook to run from a clean environment.

Because the original processed CSV was not directly available, this project performs an adapted reproduction rather than an exact reproduction. With the lecturer's approval, the experimental part of this work uses `dataset_phishing.csv`, a separate phishing URL dataset. The adapted notebook applies the same general idea as the original project: phishing URL detection using URL and webpage-based features with machine learning models.

In the adapted reproduction, the notebook was made more reproducible by using a clear dataset path, documenting the preprocessing steps, encoding the target variable, removing irrelevant and redundant features, training multiple models, and saving result files. The adapted repository also includes the dataset, notebook, requirements file, results, and report structure so that another user can rerun the experiment more easily.

Overall, the original project is partially reproducible at the conceptual level because the README explains the methodology and the notebook provides code. However, it is not fully reproducible at the experimental level because the exact processed dataset and some preprocessing steps are missing. Therefore, the original results should be treated carefully, and the adapted experiment in this project serves as an independent validation of the general methodology rather than an exact replication of the original reported results.



main





1 Branch



0 Tags



Go to file



Code





RimTouny

Update README.md

54fb5b8 · 3 years ago

 13 Commits

 LICENSE	Initial commit	3 years ago
 Phishing-Attack-Detection.ipynb	Add files via upload	3 years ago
 README.md	Update README.md	3 years ago

Figure 4: Original GitHub repository structure. The repository contains the README file and the notebook, but the processed CSV dataset is not included. This limits reproducibility because the notebook cannot be executed from a clean clone of the repository without the missing dataset.

2.Data Loading

```
data_path = r"E:\data\data_Features (2).csv"
Dataset = pd.read_csv(data_path, error_bad_lines=False)
Dataset
```

Figure 5: Local data path in the original notebook. The original notebook loads the processed dataset from a local Windows path, `E:\data\data\_Features (2).csv`, which is specific to the author's computer and is not available in the repository.

5. Experimental Results

The experimental part of this project was performed as an adapted reproduction of the original phishing URL detection methodology. Because the original processed CSV file was not directly available, the experiment was conducted using the approved `dataset_phishing.csv` dataset. The goal was to test the same general idea as the original project: using URL and webpage-based features to classify URLs as either phishing or legitimate.

The dataset used in the adapted experiment contained 11,430 URL samples and 89 original columns. The target column was `status`, with two classes: `legitimate` and `phishing`. The dataset was perfectly balanced, with 5,715 legitimate URLs and 5,715 phishing URLs. The target values were encoded numerically, where `legitimate` was mapped to 0 and `phishing` was mapped to 1.

Several modifications were introduced compared to the original source. First, the dataset was replaced with the approved `dataset_phishing.csv` file because the original processed dataset was unavailable. Second, the notebook was reorganized into a complete and executable workflow, including data loading, data inspection, EDA, preprocessing, feature engineering, model training, evaluation, and error analysis. Third, irrelevant columns such as `url` and `status` were removed from the feature matrix, and the encoded target label was also excluded from the input features to avoid target leakage. Constant or non-informative columns were removed, and the final feature matrix contained 81 numerical features.

The data was split into training and testing sets using an 80/20 stratified split. Stratification was used to preserve the same ratio of legitimate and phishing URLs in both the training and test sets. This is important because class balance affects the interpretation of evaluation metrics such as accuracy, precision, recall, and F1-score.

Several machine learning models were trained and compared. The models included Logistic Regression, Decision Tree, Random Forest, Extra Trees, Gradient Boosting, and K-Nearest Neighbors. Logistic Regression and K-Nearest Neighbors were trained using `StandardScaler` inside a pipeline because these models are sensitive to feature scale. Tree-based models such as Decision Tree, Random Forest, Extra Trees, and Gradient Boosting did not require scaling.

The models were evaluated using several classification metrics: accuracy, precision, recall, F1-score, F2-score, Matthews Correlation Coefficient (MCC), ROC-AUC, and confusion matrix values. Accuracy measures the overall percentage of correct predictions, but it is not sufficient by itself for cybersecurity problems. Precision measures how many URLs predicted as phishing were truly phishing. Recall measures how many real phishing URLs were successfully detected. F1-score balances precision and recall, while F2-score gives more weight to recall, which is important because missing a phishing URL is dangerous. MCC and ROC-AUC provide additional robust measures of classification quality.

The best-performing model according to F1-score was Random Forest. It achieved an accuracy of 0.9624, phishing precision of 0.9576, phishing recall of 0.9676, and phishing F1-score of 0.9626. The confusion matrix showed 1,094 true legitimate predictions, 1,106 true phishing predictions, 49 false positives, and 37 false negatives. This means that the model correctly detected most phishing URLs, but still missed 37 phishing URLs by classifying them as legitimate.

The model comparison showed that ensemble-based tree models performed better than the simpler models. Random Forest achieved the best overall result, followed by Extra Trees and Gradient Boosting. Logistic Regression and K-Nearest Neighbors also performed well, but their results were slightly lower. This suggests

that non-linear tree-based models were better able to capture complex relationships between URL/webpage features and the phishing label.

A cross-validation check was also performed to test the stability of the final model. The cross-validation results were consistent with the train/test split results, with mean accuracy of approximately 0.9640, mean F1-score of approximately 0.9640, and mean ROC-AUC of approximately 0.9932. This supports the conclusion that the model’s performance was stable across different folds of the data.

Overall, the experimental results support the general idea that machine learning can be effective for phishing URL detection when meaningful URL and webpage-based features are available. However, the results should still be interpreted carefully because the experiment was performed on a balanced dataset and did not include temporal validation. In real-world deployment, phishing URLs may be less frequent than legitimate URLs, and phishing techniques may change over time.

	model	accuracy	precision	recall	f1	f2	mcc	roc_auc	tn	fp	fn	tp
2	Random Forest	0.962380	0.957576	0.967629	0.962576	0.965602	0.924810	0.993394	1094	49	37	1106
3	Extra Trees	0.958880	0.958880	0.958880	0.958880	0.958880	0.917760	0.991466	1096	47	47	1096
4	Gradient Boosting	0.950569	0.945502	0.956255	0.950848	0.954085	0.901196	0.990326	1080	63	50	1093
1	Decision Tree	0.938758	0.937228	0.940507	0.938865	0.939850	0.877521	0.951891	1071	72	68	1075
5	K-Nearest Neighbors	0.939633	0.953930	0.923885	0.938667	0.929741	0.879702	0.978844	1092	51	87	1056
0	Logistic Regression	0.936133	0.939207	0.932633	0.935909	0.933941	0.872287	0.981422	1074	69	77	1066

Figure 6: Model comparison results. The table compares the trained classifiers using accuracy, precision, recall, F1-score, F2-score, MCC, ROC-AUC, and confusion matrix values. Random Forest achieved the best F1-score among the tested models.

Best model based on F1 score: Random Forest				
Classification report:				
	precision	recall	f1-score	support
legitimate	0.9673	0.9571	0.9622	1143
phishing	0.9576	0.9676	0.9626	1143
accuracy			0.9624	2286
macro avg	0.9624	0.9624	0.9624	2286
weighted avg	0.9624	0.9624	0.9624	2286

Figure 7: Classification report of the best model. Random Forest achieved strong performance on both legitimate and phishing classes, with an overall accuracy of 0.9624 and phishing recall of 0.9676.

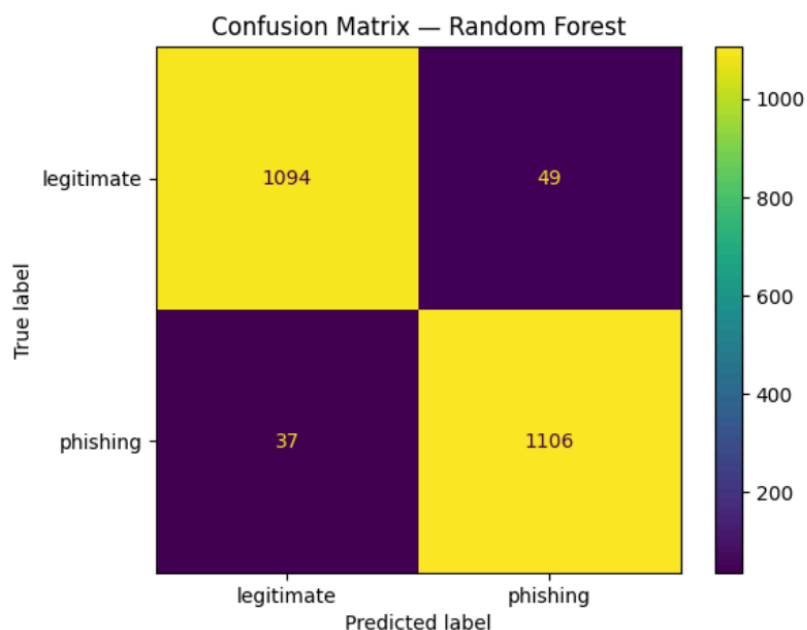


Figure 8: Confusion matrix of the Random Forest model. The model correctly classified 1,094 legitimate URLs and 1,106 phishing URLs. It produced 49 false positives, where legitimate URLs were incorrectly classified as phishing, and 37 false negatives, where phishing URLs were incorrectly classified as legitimate.

Out of 2,286 URLs in the test set, the Random Forest model correctly classified 2,200 URLs and misclassified 86 URLs. Specifically, it correctly classified 1,094 legitimate URLs and 1,106 phishing URLs. The model produced 49 false positives and 37 false negatives.

The false negatives are the most important errors in this cybersecurity context because they represent phishing URLs that were incorrectly predicted as legitimate.

	mean	std
accuracy	0.963955	0.004406
precision	0.962705	0.006701
recall	0.965354	0.005410
f1	0.964009	0.004355
roc_auc	0.993162	0.001257

Figure 9: Cross-validation summary. The cross-validation results show stable model performance across multiple folds. The final model achieved mean accuracy of 0.9640, mean precision of 0.9627, mean recall of 0.9654, mean F1-score of 0.9640, and mean ROC-AUC of 0.9932.

6. Conclusions

This project critically evaluated and adapted a published GitHub project on phishing URL detection using machine learning. The original project proposed using URL-based features and several classification models to distinguish between phishing and benign URLs. The general idea was found to be relevant and meaningful because phishing URLs often contain structural, statistical, or reputation-based patterns that can be captured by machine learning models.

The main experimental finding of the adapted reproduction was that machine learning performed well on the approved `dataset_phishing.csv` dataset. The best-performing model was Random Forest, which achieved an accuracy of 0.9624, phishing recall of 0.9676, phishing F1-score of 0.9626, and ROC-AUC of 0.9934. These results show that URL and webpage-based features can provide useful predictive signals for phishing detection.

The confusion matrix showed that the Random Forest model correctly classified 1,094 legitimate URLs and 1,106 phishing URLs. However, the model also produced 49 false positives and 37 false negatives. In the cybersecurity context, the false negatives are especially important because they represent phishing URLs that were incorrectly classified as legitimate. This means that even a strong model can still miss dangerous URLs, and error analysis is essential.

One important lesson learned is that accuracy alone is not enough for evaluating cybersecurity models. A model may achieve high accuracy but still create risk if it misses phishing URLs. Therefore, metrics such as recall, F1-score, F2-score, MCC, ROC-AUC, and the confusion matrix provide a more complete understanding of model performance. In phishing detection, recall is especially important because it measures how many real phishing URLs were successfully detected.

Another lesson learned is that reproducibility is a major part of evaluating data science projects. The original repository included a notebook and README file, but the processed CSV file used by the original notebook was not directly available. The notebook also loaded the dataset from a local Windows path. This limited the ability to reproduce the original experiment exactly. As a result, this project performed an adapted reproduction using an approved phishing dataset.

The proposed solution has several strengths. First, the problem is important and clearly connected to cybersecurity. Second, the use of URL-based and webpage-based features is meaningful because phishing attacks often leave detectable patterns in URL structure, reputation indicators, and webpage behavior. Third, comparing multiple models is a good methodology because it avoids relying on a single classifier. Finally, the adapted notebook improved reproducibility by documenting the workflow, using a clear dataset, saving results, and including model evaluation and error analysis.

However, there are also weaknesses and limitations. The original project is not fully reproducible because the exact processed dataset is missing. Some feature engineering steps may be difficult to verify, and certain features may introduce a risk of data leakage if they are computed using information that is too close to the target label. In the adapted experiment, the dataset was balanced, which is useful for model training, but it may not represent real-world phishing prevalence, where legitimate URLs are usually much more common than phishing URLs. Another limitation is the lack of temporal features, which prevented testing whether the model generalizes to future phishing campaigns.

Future improvements could include using a time-based train/test split if timestamp data becomes available. This would help test whether the model can detect newer phishing attacks rather than only similar examples from the

same dataset. Additional features could also improve performance, such as domain age, number of redirects, suspicious keywords, login form indicators, SSL certificate information, and webpage content features. Future work should also test the model on an external dataset to evaluate generalization beyond the current data.

Overall, the results support the general conclusion that machine learning can be useful for phishing URL detection when meaningful features are available. However, the conclusions should be interpreted carefully because reproducibility, data leakage, dataset representativeness, and temporal generalization are important limitations. The project demonstrates that a strong cybersecurity machine learning solution requires not only high numerical performance, but also reliable data, transparent preprocessing, appropriate metrics, and careful error analysis.

Summing It Up

This notebook evaluated the idea of phishing URL detection using machine learning, based on the selected GitHub project “Phishing Attack Detection using Machine Learning”. The original project addresses the problem of classifying URLs as phishing or benign using URL-based features and machine learning classifiers.

The selected problem is important because phishing is a major cybersecurity threat. A phishing URL can trick users into entering passwords, private information, or financial details into a fake website. Therefore, detecting phishing URLs automatically can reduce risk and support cybersecurity defense systems.

The original project used a large URL dataset created by combining several sources and proposed a complete machine learning pipeline. The methodology included URL feature extraction, exploratory data analysis, data cleaning, feature selection, handling class imbalance with SMOTE, training multiple classifiers, and using ensemble methods such as stacking and soft voting.

The critical evaluation showed that the original approach is relevant and appropriate for the phishing detection problem. URL-based features are meaningful because phishing URLs often contain suspicious patterns, such as unusual length, many digits or special characters, redirection, shortened links, weak reputation, or suspicious domain structure.

However, the original project also has important limitations. The main limitation is reproducibility. The processed CSV file used by the original notebook was not directly available in the repository, and the notebook loaded the dataset from a local Windows path. Therefore, the original experiment could not be reproduced exactly. This suggests that the original results should be treated carefully.

Because the original processed dataset was unavailable, this notebook performed an adapted reproduction using the approved `dataset_phishing.csv` dataset. The dataset contained 11,430 URL samples, balanced between 5,715 legitimate URLs and 5,715 phishing URLs. The target was encoded numerically, irrelevant columns were removed, preprocessing was applied, and the final feature matrix contained 81 numerical features.

Several models were trained and compared: Logistic Regression, Decision Tree, Random Forest, Extra Trees, Gradient Boosting, and K-Nearest Neighbors. The best model was Random Forest, which achieved accuracy of 0.9624, phishing recall of 0.9676, phishing F1-score of 0.9626, and ROC-AUC of 0.9934. Cross-validation also showed stable performance across multiple folds.

The reproduction study supports the general claim that machine learning can be useful for phishing URL detection. The results show that URL and webpage-based features can provide strong predictive signals, especially when using ensemble tree-based models such as Random Forest. However, the results do not fully prove the original project’s claims because the exact original dataset was unavailable and the adapted experiment used a different approved dataset.

The most important insight from this analysis is that strong numerical performance is not enough in cybersecurity. A phishing detection model must also be evaluated using appropriate metrics such as recall, F1-score, F2-score, MCC, ROC-AUC, and confusion matrix analysis. False negatives are especially dangerous because they represent phishing URLs that were incorrectly classified as legitimate.

I would recommend using this type of project as a basis for similar phishing detection problems, but only with improvements. The dataset and preprocessing pipeline should be fully available, the model should be tested on

external datasets, and temporal validation should be added to evaluate whether the model generalizes to future phishing campaigns.

The final conclusion is that the selected project presents a meaningful machine learning approach to phishing detection, but its original evidence is limited by reproducibility issues and possible methodological weaknesses. The adapted reproduction supports the general idea while showing the importance of transparent data, careful feature engineering, robust evaluation, and cybersecurity-aware error analysis.